

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: *Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Technical Presentation

DURATION: 2HRS

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

- Bellman Principle of Optimality
- Policy Evaluation and Policy Improvement
- Value Iteration vs. Policy Iteration
- Industrial applications

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

- Monte Carlo Prediction and Control
- TD(0), SARSA, and Q-Learning
- Sample efficiency
- Performance comparison through experiments

3. Deep Q-Networks for Intelligent Decision-Making

- DQN Architecture
- Experience Replay
- Target Networks
- Industrial applications in robotics and autonomous systems

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

- Double DQN (DDQN)
- Dueling DQN
- Prioritized Experience Replay (PER)
- Performance comparison using benchmark environment

5. Policy-Based Reinforcement Learning for Continuous Control Problems

- Vanilla Policy Gradient
- REINFORCE Algorithm
- Stochastic Policy Search
- Applications in robotics and autonomous driving
- Comparative analysis of Value-Based, Policy-Based, and Model-Based RL approaches

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

1. Introduction

Reinforcement Learning (RL) is a machine learning technique in which an agent learns by interacting with an environment. At every step, the agent selects an action and receives a reward.

Dynamic Programming (DP) provides methods for solving RL problems when the environment's transition probabilities and rewards are known.

The main DP techniques are:

- Bellman Principle of Optimality
- Policy Evaluation
- Policy Improvement
- Policy Iteration
- Value Iteration

2. Bellman Principle of Optimality

The Bellman Principle of Optimality states:

An optimal policy has the property that whatever the initial state and first action are, the remaining decisions must constitute an optimal policy for the resulting state.

In simple words:

Optimal solution = immediate reward + optimal solution for the next state.

The Bellman optimality equation is:

Where:

- = optimal value of state
- = reward
- = discount factor
- = transition probability
- = value of next state

3. Policy Evaluation

A policy tells the agent which action to choose in each state.

Policy evaluation calculates the value of following a particular policy.

Bellman expectation equation:

Steps

1. Initialize values randomly or to zero.
2. Follow the given policy.
3. Calculate the value of each state.
4. Repeat until values converge.

4. Policy Improvement

After evaluating a policy, we try to find better actions.

The agent selects:

If the new policy produces better values, replace the old policy.

This process continues until no improvement is possible.

5. Policy Iteration

Policy iteration alternates between:

Policy Evaluation → Policy Improvement

Algorithm

Initialize policy π

Repeat:

 Evaluate π

 Improve π

Until policy does not change

Return optimal policy

6. Value Iteration

Value iteration directly searches for optimal state values.

After convergence:

7. Value Iteration vs Policy Iteration

Feature	Value Iteration	Policy Iteration
Main operation	Value update	Policy evaluation + improvement
Policy	Derived at end	Maintained throughout
Iterations	Usually more	Usually fewer
Computation per iteration	Lower	Higher
Suitable for	Large state spaces	Smaller/medium problems

8. Industrial Applications

Dynamic programming concepts are used in:

- Robotics: path planning and movement
- Autonomous vehicles: route and decision planning
- Finance: portfolio optimization
- Supply chain: inventory management
- Manufacturing: production scheduling
- Energy: smart-grid optimization
- Telecommunications: resource allocation

9. Conclusion

Dynamic Programming provides a mathematical foundation for sequential decision-making. Policy iteration and value iteration can find optimal policies when the environment model is known. These concepts also form the foundation for many modern reinforcement learning algorithms.

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

1. Introduction

Monte Carlo (MC) and Temporal-Difference (TD) learning are model-free reinforcement learning methods.

They learn from experience without requiring complete knowledge of the environment.

The major difference is:

- Monte Carlo: learns after an episode finishes.
- TD: learns after every step.

3. Monte Carlo Prediction

Monte Carlo estimates the value of a state using the actual return obtained after an episode.

Return:

The value is updated using:

Advantages

- Simple
- Does not require environment model
- Uses actual returns

Disadvantage

The episode must usually finish before updating.

4. Monte Carlo Control

MC control learns the best policy by:

1. Generating episodes.
2. Estimating action values.
3. Improving the policy.
4. Repeating the process.

It often uses ϵ -greedy exploration.

5. TD(0)

TD learning updates values before the episode ends.

TD target:

Update:

The term:

is called the TD error.

6. SARSA

SARSA is an on-policy TD control algorithm.

SARSA stands for:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ State $\rightarrow$ Action

Update:

It learns the value of the policy that the agent actually follows.

7. Q-Learning

Q-learning is an off-policy algorithm.

Update:

It learns toward the best possible next action.

8. Comparison

Feature	Monte Carlo	TD(0)	SARSA	Q-Learning
Learning	End of episode	Every step	Every step	Every step
Model required	No	No	No	No
Target	Actual return	Estimated return	Next selected action	Best next action
Policy	Usually on-policy	On-policy	On-policy	Off-policy
Sample efficiency	Lower	Higher	Higher	Higher

9. Sample Efficiency

Sample efficiency refers to how much useful learning an algorithm obtains from each interaction.

TD methods are generally more sample-efficient because they can learn after every transition, while Monte Carlo methods typically wait until the complete episode is available.

10. Experimental Comparison

A simple experiment can use environments such as:

- FrozenLake
- CartPole
- CliffWalking

Performance can be measured using:

- Average reward
- Number of episodes
- Convergence speed
- Success rate

Typical observation:

TD/Q-learning → faster updates and better sample efficiency

Monte Carlo → simpler but requires complete episodes

11. Conclusion

Monte Carlo methods learn from complete returns, while TD methods combine actual rewards with estimated future values. SARSA is on-policy, whereas Q-learning is off-policy. TD algorithms are generally preferred when continuous, step-by-step learning is required.

3. Deep Q-Networks for Intelligent Decision-Making

1. Introduction

A Deep Q-Network (DQN) combines:

- Reinforcement Learning
- Deep Neural Networks

Traditional Q-learning stores Q-values in a table. This becomes impractical when the state space is very large.

DQN uses a neural network to approximate:

2. DQN Architecture

A typical DQN contains:

Environment



State



Neural Network



Q-values for actions



Select Action



Reward + Next State

The neural network receives the state and outputs Q-values for possible actions.

The action is generally selected using an ϵ -greedy policy.

3. Q-Learning Target

The target is:

The network minimizes the loss:

where θ represents neural network parameters.

4. Experience Replay

Experience replay stores previous experiences in a memory buffer.

Each experience is:

The agent randomly samples mini-batches from this memory.

Benefits

- Reduces correlation between consecutive experiences
- Improves training stability
- Reuses past experiences
- Improves sample efficiency

5. Target Network

DQN uses two networks:

Online Network

Updated frequently during training.

Target Network

Updated less frequently.

The target network calculates stable target values:

This prevents the target from changing too rapidly.

6. DQN Training Process

Initialize online network

Initialize target network

Initialize replay memory

Observe state

Select action

Perform action

Store experience

Sample random mini-batch

Calculate target

Update online network

Periodically update target network

Repeat

7. Advantages

- Handles large state spaces
- Learns directly from high-dimensional inputs
- Can process images
- Does not require manually designed policies

8. Industrial Applications

Robotics

DQN can help robots learn:

- Object manipulation
- Navigation
- Task selection
- Collision avoidance

Autonomous Systems

Applications include:

- Autonomous driving decisions
- Lane-change decisions
- Traffic management
- Navigation

Other applications

- Data-center resource management
- Smart energy systems
- Industrial control

- Game AI

9. Limitations

- Requires large amounts of training data
- Training can be unstable
- Computationally expensive
- Standard DQN is mainly suitable for discrete actions

10. Conclusion

DQN combines deep learning with Q-learning to solve large-scale decision-making problems. Experience replay and target networks are two key techniques that make DQN more stable and effective.

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

1. Introduction

Standard DQN can suffer from problems such as:

- Overestimation of Q-values
- Poor representation of state values
- Inefficient experience sampling

Three important improvements are:

1. Double DQN
2. Dueling DQN
3. Prioritized Experience Replay

2. Double DQN

Standard DQN uses:

This can cause overestimation bias.

Double DQN separates:

- Action selection
- Action evaluation

The target becomes:

Benefit

Reduces overestimation of action values.

3. Dueling DQN

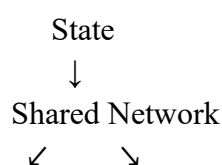
Dueling DQN separates the Q-value into:

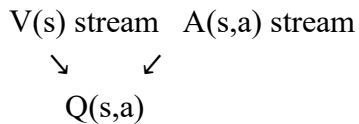
State Value

Advantage

The final Q-value is approximately:

The network architecture is:





Benefit

It can learn which states are valuable even when the exact action has little effect.

4. Prioritized Experience Replay

Normal DQN randomly samples experiences.

PER gives higher probability to experiences with large TD errors.

Priority is commonly related to:

where:

Experiences with larger errors are more likely to be selected.

Benefits

- Faster learning
- Focuses on important experiences
- Improves sample efficiency

5. Comparison

Feature	DQN	Double DQN	Dueling DQN	PER
Main problem addressed	Basic Q-learning	Overestimation	State/action representation	Inefficient sampling
Main technique	Replay + target	Separate selection/evaluation	Value + advantage streams	Priority-based replay
Stability	Good	Better	Better	Better
Sample efficiency	Moderate	Moderate	Moderate	High

6. Benchmark Environments

These algorithms can be compared using:

- CartPole
- LunarLander
- Atari games
- MountainCar

Performance metrics:

- Average reward
- Number of episodes
- Training time
- Convergence speed
- Stability

A typical experiment can plot:

Episode Number → X-axis

Average Reward → Y-axis

The algorithm achieving higher reward with fewer episodes is considered more efficient.

7. Practical Comparison

Double DQN

Best when Q-value overestimation is the major problem.

Dueling DQN

Best when many actions have similar effects in a state.

PER

Best when some experiences are much more informative than others.

These techniques can also be combined into more advanced architectures.

8. Conclusion

Advanced DQN algorithms improve different weaknesses of standard DQN. Double DQN reduces overestimation, Dueling DQN improves value representation, and PER improves experience selection. Together, these techniques can significantly improve learning performance.

5. Policy-Based Reinforcement Learning for Continuous Control Problems

1. Introduction

Reinforcement learning methods can be divided into:

- Value-based methods
- Policy-based methods
- Actor-Critic methods

Policy-based methods directly learn a policy:

Instead of learning Q-values first.

They are particularly useful for continuous action spaces.

2. Policy Gradient

The objective is to maximize expected cumulative reward:

The policy gradient is:

The policy parameters are updated using:

3. Vanilla Policy Gradient

Vanilla Policy Gradient directly adjusts the policy parameters in the direction that increases expected reward.

Steps

1. Initialize policy parameters.
2. Run the policy in the environment.
3. Collect states, actions and rewards.
4. Calculate returns.
5. Calculate policy gradient.
6. Update policy.
7. Repeat.

4. REINFORCE Algorithm

REINFORCE is a fundamental Monte Carlo policy-gradient algorithm.

The update rule is:

Algorithm

Initialize policy parameters θ

Repeat:

 Generate an episode

 Calculate return G

 For each state-action pair:

 Update θ using G

Until convergence

5. Stochastic Policy Search

A stochastic policy selects actions according to a probability distribution.

For example:

For continuous control, the policy may output parameters of a Gaussian distribution:

This allows the agent to explore different continuous actions.

6. Why Policy-Based Methods Are Useful for Continuous Control

Consider a robot arm.

Its motor movement could be:

or

There are infinitely many possible actions.

A Q-table would struggle with this.

Policy-based methods can directly generate continuous actions.

7. Applications in Robotics

Policy-gradient methods can be used for:

- Robot arm control
- Walking robots
- Grasping
- Object manipulation
- Robot navigation
- Motor control

Example:

Camera/Sensors



State



Policy Network



Continuous Action



Robot



Reward



Policy Update

8. Applications in Autonomous Driving

Policy-based RL can be used for:

- Steering control
- Acceleration
- Braking
- Lane changing
- Path planning
- Decision-making

For example, the policy can directly produce:
instead of selecting from only a few discrete actions.

9. Advantages

- Naturally handles continuous actions
- Directly optimizes the policy
- Can learn stochastic policies
- Suitable for complex control problems

10. Limitations

- High variance
- Can require many samples
- Training can be unstable
- Sensitive to learning rate
- Basic REINFORCE is often less sample-efficient than modern actor-critic methods

11. Conclusion

Policy-based reinforcement learning directly optimizes the agent's policy and is highly suitable for continuous control. Vanilla Policy Gradient and REINFORCE provide the foundation, while stochastic policy search enables exploration of continuous action spaces. These methods have important applications in robotics, autonomous vehicles, and industrial control systems.

Code of Conduct:

*I **Kandukuri Kavya-192425283** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student

Kandukuri Kavya-192425283

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

